

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Санкт-Петербургский национальный исследовательский университет
информационных технологий, механики и оптики»**



ITMO UNIVERSITY

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №4

«Анализ трафика компьютерных сетей с помощью утилиты Wireshark»

по дисциплине

«КОМПЬЮТЕРНЫЕ СЕТИ»:

Выполнил:

Ястребов-Амирханов Алекси

ГРУППА: Р3332

ПРЕПОДАВАТЕЛЬ: Алиев Тауфик Измайлович

Санкт-Петербург,

2026

Оглавление

Задание.....	2
Цель работы	2
Вариант	2
Анализ трафика утилиты ping	3
Выполнение.....	3
Ответы на вопросы:.....	5
Анализа трафика утилиты tracert (traceroute)	7
Выполнение.....	7
Ответы на вопросы:.....	8
Анализ HTTP-трафика	11
Выполнение.....	11
Анализ ARP-трафика.....	13
Выполнение.....	13
Ответы на вопросы:.....	14
Вывод	16

Задание

Цель работы

Целью данной лабораторной работы является изучение структуры протокольных блоков данных, путем анализа реального трафика на компьютере студента с помощью бесплатно распространяемой утилиты Wireshark.

В процессе выполнения домашнего задания выполняются наблюдения за передаваемым трафиком с компьютера пользователя в Интернет и в обратном направлении.

Применение специализированной утилиты Wireshark позволяет наблюдать структуру передаваемых кадров, пакетов и сегментов данных различных сетевых протоколов. При выполнении УИР рекомендуется выполнить анализ последовательности команд и определить назначение служебных данных, используемых для организации обмена данными в протоколах: ARP, DNS, FTP, HTTP, DHCP.

Вариант

Для выполнения лабораторной работы будут представлены пункты 4.1, 4.2, 4.3, 4.5.

Сайт для анализа трафика – yaa.ms.

Анализ трафика утилиты ping

Выполнение

В командной строке поочередно с увеличением размера будем отправлять пакеты через утилиту ping на сайт yaa.ms. Формат команды:

ping -l <количество байт> -n 1 yaa.ms

```
((((ip6.src == 64:f9b::57ec:105a) && ((ip6.dst == 2a03:d000:85:647c:bc64:14f0:eff2:82d3)) || ((ip6.src == 2a03:d000:85:647c:bc64:14f0:eff2:82d3) && (ip6.dst == 64:f9b::57ec:105a))))(((ip.src == 87.236.16.90) && (ip.dst == 192.168.11.109)) || ((ip.src == 192.168.11.109) && (ip.dst == 87.236.16.90)))) && ((icmp.resp_not_found) && ((icmp.no_resp))
```

Опция **-n** нужна для того, чтобы отправлять один пакет, так как сама по себе утилита по умолчанию отправляет 4 пакета.

Применим данную команду несколько раз для случаев, когда размер у пакета будет 100, 200, 500, 1000, 2000, 5000, 10000 байт.

Source	Destination	Protocol	Length	Info
192.168.11.109	87.236.16.90	ICMP	142	Echo (ping) request id=0x0001, seq=113/28928, ttl=64 (reply in 5734)
87.236.16.90	192.168.11.109	ICMP	146	Echo (ping) reply id=0x0001, seq=113/28928, ttl=56 (request in 5733)
192.168.11.109	87.236.16.90	ICMP	242	Echo (ping) request id=0x0001, seq=114/29184, ttl=64 (reply in 6931)
87.236.16.90	192.168.11.109	ICMP	246	Echo (ping) reply id=0x0001, seq=114/29184, ttl=56 (request in 6930)
192.168.11.109	87.236.16.90	ICMP	342	Echo (ping) request id=0x0001, seq=115/29440, ttl=64 (reply in 7127)
87.236.16.90	192.168.11.109	ICMP	346	Echo (ping) reply id=0x0001, seq=115/29440, ttl=56 (request in 7126)
192.168.11.109	87.236.16.90	ICMP	542	Echo (ping) request id=0x0001, seq=116/29696, ttl=64 (reply in 7204)
87.236.16.90	192.168.11.109	ICMP	546	Echo (ping) reply id=0x0001, seq=116/29696, ttl=56 (request in 7203)
192.168.11.109	87.236.16.90	ICMP	1042	Echo (ping) request id=0x0001, seq=117/29952, ttl=64 (reply in 7334)
87.236.16.90	192.168.11.109	ICMP	1046	Echo (ping) reply id=0x0001, seq=117/29952, ttl=56 (request in 7329)
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=e553) [Reassembled in #7650]
192.168.11.109	87.236.16.90	ICMP	562	Echo (ping) request id=0x0001, seq=118/30208, ttl=64 (reply in 7652)
87.236.16.90	192.168.11.109	IPv4	1518	Fragmented IP protocol (proto=ICMP 1, off=0, ID=9b6e) [Reassembled in #7652]
87.236.16.90	192.168.11.109	ICMP	562	Echo (ping) reply id=0x0001, seq=118/30208, ttl=56 (request in 7650)
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=e554) [Reassembled in #7750]
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e554) [Reassembled in #7750]
192.168.11.109	87.236.16.90	ICMP	82	Echo (ping) request id=0x0001, seq=119/30464, ttl=64 (no response found!)
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=e555) [Reassembled in #8195]
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e555) [Reassembled in #8195]
192.168.11.109	87.236.16.90	ICMP	82	Echo (ping) request id=0x0001, seq=120/30720, ttl=64 (reply in 8193)
87.236.16.90	192.168.11.109	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=9bce) [Reassembled in #8198]
87.236.16.90	192.168.11.109	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=9bce) [Reassembled in #8198]
87.236.16.90	192.168.11.109	ICMP	82	Echo (ping) reply id=0x0001, seq=120/30720, ttl=56 (request in 8195)
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=e556) [Reassembled in #8331]
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e556) [Reassembled in #8331]
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=e556) [Reassembled in #8331]
192.168.11.109	87.236.16.90	ICMP	602	Echo (ping) request id=0x0001, seq=121/30976, ttl=64 (no response found!)
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=e557) [Reassembled in #8407]
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e557) [Reassembled in #8407]
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=e557) [Reassembled in #8407]
192.168.11.109	87.236.16.90	ICMP	602	Echo (ping) request id=0x0001, seq=122/31232, ttl=64 (reply in 8411)
87.236.16.90	192.168.11.109	IPv4	1518	Fragmented IP protocol (proto=ICMP 1, off=0, ID=a5cb) [Reassembled in #8411]
87.236.16.90	192.168.11.109	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=1480, ID=a5cb) [Reassembled in #8411]
87.236.16.90	192.168.11.109	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=2960, ID=a5cb) [Reassembled in #8411]
87.236.16.90	192.168.11.109	ICMP	602	Echo (ping) reply id=0x0001, seq=122/31232, ttl=56 (request in 8407)
192.168.11.109	87.236.16.90	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=e558) [Reassembled in #8571]

Для начала опишем структуру пакета. Утилита управляет ICMP запросами и ICMP ответами. Структура:

1. Канальный уровень – Ethernet 2

Заголовок содержит:

- Destination MAC address – MAC адрес получателя.
- Source MAC address – MAC-адрес отправителя.
- Type – поле типа протокола.

2. Сетевой уровень – IP-заголовок

Заголовок содержит:

- Version
- Header Length
- Identification – идентификатор фрагмента
- Protocol – тип вложенного протокола
- Flags – указывается DF и MF
- TTL – ограничение на кол-во хопов
- Fragment offset – смещение фрагмента (если пакет был фрагментирован)
- Header Checksum – контрольная сумма заголовка
- Source IP address
- Destination IP address

3. Сетевой протокол ICMP

- Type – request или reply
- Checksum – контрольная сумма ICMP-пакета
- Identifier – уникальный ID запроса
- Seq number – номер последовательности запроса

4. Поле данных (Payload)

Примерно так выглядит структура пакета ICMP:

```
Frame 5733: Packet, 142 bytes on wire (1136 bits), 142 bytes captured (1136 bits) on interface \Device\NPF_{18B4D016-7E54-4813-AD18-0959C7D0E416}, id 0
Ethernet II, Src: LiteonTechno_86:0e:33 (94:08:53:86:0e:33), Dst: Routerboardc_a2:cf:d0 (d4:01:c3:a2:cf:d0)
  Destination: Routerboardc_a2:cf:d0 (d4:01:c3:a2:cf:d0)
  Source: LiteonTechno_86:0e:33 (94:08:53:86:0e:33)
  Type: IPv4 (0x0800)
  [Stream index: 0]
Internet Protocol Version 4, Src: 192.168.11.109, Dst: 87.236.16.90
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 128
  Identification: 0xe54e (58702)
  0000 .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x60d3 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.11.109
  Destination Address: 87.236.16.90
  [Stream index: 73]
Internet Control Message Protocol
  Type: Echo (ping) request (8)
  Code: 0
  Checksum: 0xea7c [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 113 (0x0071)
  Sequence Number (LE): 28928 (0x7100)
  [Response frame: 5734]
  Data (100 bytes)
0000 d4 01 c3 a2 cf d0 94 08 53 86 0e 33 08 00 45 00 .....S-3-E
0010 00 80 e5 4e 00 00 40 01 60 d3 c0 a8 0b 6d 57 ec ...N @ .....mJ
0020 10 5a 08 00 ea 7c 00 01 00 71 61 62 63 64 65 66 -Z...|...ghbcdef
0030 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 ghijklmn opqrstuv
0040 77 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f wabcdefg hijklmno
0050 70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 pqrstuvwxyz abcdefgh
0060 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 78 ijklmnop qrstuvw
```

Ответы на вопросы:

1. Имеет ли место фрагментация исходного пакета, какое поле на это указывает?

Фрагментация происходит, когда размер IP-пакета превышает MTU (maximum transmission unit) (обычно 1480 байт для Ethernet). Признаком фрагментации служат:

- Флаг MF (More Fragments) в IP-заголовке
- Поле Fragment Offset (смещение фрагмента)

```
▶ Frame 7649: Packet, 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on inter
▼ Ethernet II, Src: LiteonTechno_86:0e:33 (94:08:53:86:0e:33), Dst: Routerboardc_a2:cf:d0 (d4:01
  ▶ Destination: Routerboardc_a2:cf:d0 (d4:01:c3:a2:cf:d0)
  ▶ Source: LiteonTechno_86:0e:33 (94:08:53:86:0e:33)
  Type: IPv4 (0x0800)
  [Stream index: 0]
▼ Internet Protocol Version 4, Src: 192.168.11.109, Dst: 87.236.16.90
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▼ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
    .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 1500
  Identification: 0xe553 (58707)
  ▼ 001. .... = Flags: 0x1, More fragments
    0... .... = Reserved bit: Not set
    .0.. .... = Don't fragment: Not set
    ..1. .... = More fragments: Set
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x3b72 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.11.109
  Destination Address: 87.236.16.90
  [Reassembled IPv4 in frame: 7650]
  [Stream index: 73]
▶ Data (1480 bytes)
```

```
▶ Frame 7650: Packet, 562 bytes on wire (4496 bits), 562 bytes captured (4496 bits) on interface
▼ Ethernet II, Src: LiteonTechno_86:0e:33 (94:08:53:86:0e:33), Dst: Routerboardc_a2:cf:d0 (d4:01
  ▶ Destination: Routerboardc_a2:cf:d0 (d4:01:c3:a2:cf:d0)
  ▶ Source: LiteonTechno_86:0e:33 (94:08:53:86:0e:33)
  Type: IPv4 (0x0800)
  [Stream index: 0]
▼ Internet Protocol Version 4, Src: 192.168.11.109, Dst: 87.236.16.90
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▼ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
    .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 548
  Identification: 0xe553 (58707)
  ▼ 000. .... = Flags: 0x0
    0... .... = Reserved bit: Not set
    .0.. .... = Don't fragment: Not set
    ..0. .... = More fragments: Not set
  ...0 0000 1011 1001 = Fragment Offset: 1480
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x5e71 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.11.109
  Destination Address: 87.236.16.90
  ▶ [2 IPv4 Fragments (2008 bytes): #7649(1480), #7650(528)]
  [Stream index: 73]
▶ Internet Control Message Protocol
```

Заметим, что когда пакет фрагментируется, то часть данных отправляется вместе с ICMP заголовком, а остальные фрагменты чисто по протоколу IP.

2. Какая информация указывает, является ли фрагмент пакета последним или промежуточным?

- MF = 1 – промежуточный фрагмент.
- MF = 0 – последний фрагмент.

```
▼ 000. .... = Flags: 0x0
  0... .... = Reserved bit: Not set
  .0.. .... = Don't fragment: Not set
  ..0. .... = More fragments: Not set
  ...0 0000 1011 1001 = Fragment Offset: 1480
```

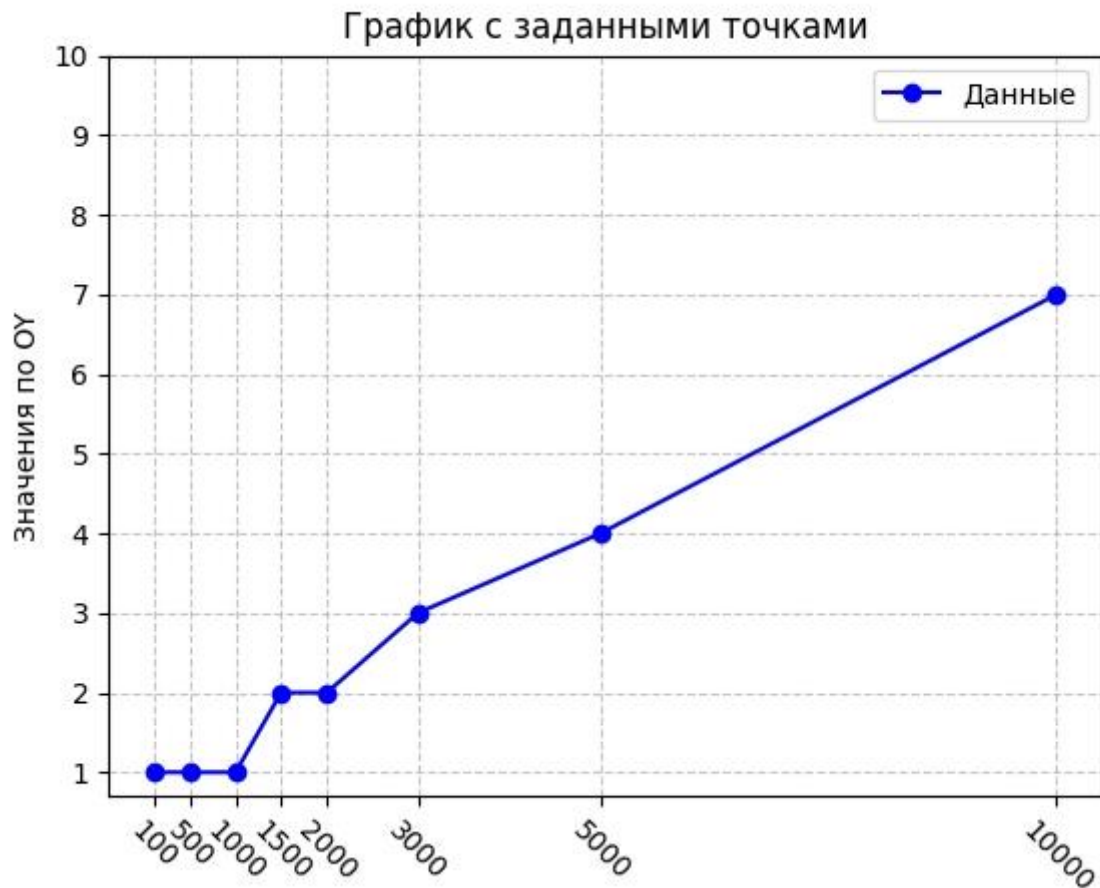
```
▼ 000. .... = Flags: 0x0
  0... .... = Reserved bit: Not set
  .0.. .... = Don't fragment: Not set
  ..0. .... = More fragments: Not set
  ...0 0000 1011 1001 = Fragment Offset: 1480
```

3. Чему равно количество фрагментов при передаче ping-пакетов?

Учитывая, что один фрагмент по MTU равен примерно 1480 байт, кол-во фрагментов будет равно размеру пакета / 1480 и округлить до верхнего целого числа.

192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e554) [Reassembled in #7750]
192.168.11.109	87.236.16.90	ICMP	82 Echo (ping) request id=0x0001, seq=119/30464, ttl=64 (no response found!)
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=0, ID=e555) [Reassembled in #8195]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e555) [Reassembled in #8195]
192.168.11.109	87.236.16.90	ICMP	82 Echo (ping) request id=0x0001, seq=120/30720, ttl=64 (reply in 8198)
87.236.16.90	192.168.11.109	IPv4	1518 Fragmented IP protocol (proto=ICMP 1, off=0, ID=9bce) [Reassembled in #8198]
87.236.16.90	192.168.11.109	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=1480, ID=9bce) [Reassembled in #8198]
87.236.16.90	192.168.11.109	ICMP	82 Echo (ping) reply id=0x0001, seq=120/30720, ttl=56 (request in 8195)
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=0, ID=e556) [Reassembled in #8331]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e556) [Reassembled in #8331]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=2960, ID=e556) [Reassembled in #8331]
192.168.11.109	87.236.16.90	ICMP	602 Echo (ping) request id=0x0001, seq=121/30976, ttl=64 (no response found!)
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=0, ID=e557) [Reassembled in #8407]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e557) [Reassembled in #8407]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=2960, ID=e557) [Reassembled in #8407]
192.168.11.109	87.236.16.90	ICMP	602 Echo (ping) request id=0x0001, seq=122/31232, ttl=64 (reply in 8411)
87.236.16.90	192.168.11.109	IPv4	1518 Fragmented IP protocol (proto=ICMP 1, off=0, ID=a5cb) [Reassembled in #8411]
87.236.16.90	192.168.11.109	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=1480, ID=a5cb) [Reassembled in #8411]
87.236.16.90	192.168.11.109	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=2960, ID=a5cb) [Reassembled in #8411]
87.236.16.90	192.168.11.109	ICMP	602 Echo (ping) reply id=0x0001, seq=122/31232, ttl=56 (request in 8407)
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=0, ID=e558) [Reassembled in #8571]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=1480, ID=e558) [Reassembled in #8571]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=2960, ID=e558) [Reassembled in #8571]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=4440, ID=e558) [Reassembled in #8571]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=5920, ID=e558) [Reassembled in #8571]
192.168.11.109	87.236.16.90	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=7400, ID=e558) [Reassembled in #8571]
192.168.11.109	87.236.16.90	ICMP	1162 Echo (ping) request id=0x0001, seq=123/31488, ttl=64 (reply in 8579)
87.236.16.90	192.168.11.109	IPv4	1518 Fragmented IP protocol (proto=ICMP 1, off=0, ID=a9f5) [Reassembled in #8579]
87.236.16.90	192.168.11.109	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=1480, ID=a9f5) [Reassembled in #8579]
87.236.16.90	192.168.11.109	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=2960, ID=a9f5) [Reassembled in #8579]
87.236.16.90	192.168.11.109	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=4440, ID=a9f5) [Reassembled in #8579]
87.236.16.90	192.168.11.109	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=5920, ID=a9f5) [Reassembled in #8579]
87.236.16.90	192.168.11.109	IPv4	1514 Fragmented IP protocol (proto=ICMP 1, off=7400, ID=a9f5) [Reassembled in #8579]

4. График: размер пакета – кол-во фрагментов.



5. Как изменить поле TTL с помощью утилиты ping?

Изменить это поле можно командой: ***ping -l 100 -n 1 -i 5 yaa.ru***

```
▼ 000. .... = Flags: 0x0
  0... .... = Reserved bit: Not set
  .0.. .... = Don't fragment: Not set
  ..0. .... = More fragments: Not set
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 5
```

6. Что содержится в поле данных ping-пакета

- Заголовок ICMP;
- Идентификатор;
- Номер последовательности;
- Содержимое.

Анализа трафика утилиты `tracert` (`tracert`)

Выполнение

Вводим в командную строку команду:

```
tracert yaa.ms
```

```
tracert -d yaa.ms
```

```
C:\Users\AsusAspire 3>tracert yaa.ms
Трассировка маршрута к yaa.ms [87.236.16.90]
с максимальным числом прыжков 30:

 1  11 ms  14 ms   4 ms  192.168.10.1
 2   8 ms   8 ms  11 ms  10.152.196.1
 3  15 ms  11 ms  10 ms  10.149.252.37
 4  13 ms   6 ms   3 ms  89.223.47.5
 5  17 ms  19 ms  15 ms  teamhost3.msk.piter-ix.net [185.0.12.221]
 6  17 ms  13 ms  15 ms  10.255.200.233
 7  32 ms  23 ms  22 ms  172.31.255.1
 8  20 ms  25 ms  20 ms  10.6.0.254
 9  27 ms  30 ms  23 ms  ssl.plasma.beget.com [87.236.16.90]

Трассировка завершена.
```

```
C:\Users\AsusAspire 3>tracert -d yaa.ms
Трассировка маршрута к yaa.ms [87.236.16.90]
с максимальным числом прыжков 30:

 1   1 ms   1 ms   1 ms  192.168.10.1
 2   *     *    10 ms  10.152.196.1
 3   5 ms   2 ms  13 ms  10.149.252.37
 4  36 ms  14 ms  40 ms  89.223.47.5
 5  11 ms  11 ms  12 ms  185.0.12.221
 6  33 ms  36 ms  47 ms  10.255.200.233
 7  19 ms  21 ms  21 ms  172.31.255.1
 8  31 ms   *    34 ms  10.6.0.254
 9  22 ms  26 ms  25 ms  87.236.16.90

Трассировка завершена.
```

Данная утилита также пользуется протоколом ICMP, поэтому разбирать его структуру мы не будем. Но, помимо этого, утилита `tracert` отправляет DNS-пакеты. DNS – это протокол, который переводит доменные имена в IP-адреса, которые понятны компьютерам. С помощью ключа `-d` можно сделать так, чтобы DNS пакеты отправлялись уже после построения маршрута, так как они не несут в себе важный функционал.

Вот структура DNS пакета:

```
▼ Domain Name System (query)
  Transaction ID: 0x0002
  ▼ Flags: 0x0100 Standard query
    0... .. = Response: Message is a query
    .000 0... .. = Opcode: Standard query (0)
    .... ..0. .... = Truncated: Message is not truncated
    .... ..1 .... = Recursion desired: Do query recursively
    .... ..0... .. = Z: reserved (0)
    .... ..0 .... = Non-authenticated data: Unacceptable
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▼ yaa.ms: type A, class IN
      Name: yaa.ms
      [Name Length: 6]
      [Label Count: 2]
      Type: A (1) (Host Address)
      Class: IN (0x0001)
      [Response In: 4364]
```

Заголовок размером 12 байт содержит:

- ID – уникальный ID-запроса;
- Flags – ошибки, авторитетность, тип запроса/ответа;

- QDCOUNT – кол-во запросов;
- ANCOUNT – кол-во ответов;
- INSCOUNT – кол-во записей авторитетных серверов;
- ARCOUNT – кол-во дополнительных записей.

Раздел вопросов, который содержит доменное имя, которое мы запрашиваем.

Раздел ответов, который содержит IP-адрес в ответ на запрос.

Ответы на вопросы:

1. Сколько байт содержится в заголовке IP? Сколько содержится в поле данных?

Заголовок IP обычно составляет 20 байт для IPv4.

```

Internet Protocol Version 4, Src: 192.168.11.109, Dst: 192.168.10.1
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 52
    Identification: 0x5f9c (24476)
  ▸ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
  
```

Поле данных – это содержимое, инкапсулированное в IP-пакете, ICMP пакета.

У ICMP заголовок равен 8 байт, а сами данные 64 байта. Следовательно, данные 72 байта. IP-заголовок 20 байт.

2. Как и почему изменяется поле TTL в следующих ICMP-пакетах tracert?

Утилита tracert посылает ICMP-пакеты с увеличивающимся TTL, начиная с 1. Каждый маршрутизатор уменьшает TTL на 1. Когда TTL становится 0 – маршрутизатор отбрасывает пакет и отправляет обратно ICMP Time Exceeded. Это позволяет tracert определить каждый узел на пути. TTL изменяется поэтапно, чтобы каждый узел по очереди откликнулся, и таким образом строится маршрут.

No.	Time	Source	Destination	Protocol	Length	Info
259	9.094554000	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=317/15617, ttl=1 (no response found!)
294	12.650196500	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=318/15873, ttl=1 (no response found!)
295	12.657258400	192.168.10.1	192.168.11.109	ICMP	138	Time-to-live exceeded (Time to live exceeded in transit)
296	12.659235200	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=319/16129, ttl=1 (no response found!)
297	12.674142100	192.168.10.1	192.168.11.109	ICMP	138	Time-to-live exceeded (Time to live exceeded in transit)
604	22.721849900	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=320/16385, ttl=2 (no response found!)
605	22.741229400	10.152.196.1	192.168.11.109	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
606	22.743935300	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=321/16641, ttl=2 (no response found!)
607	22.763835100	10.152.196.1	192.168.11.109	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
608	22.765123400	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=322/16897, ttl=2 (no response found!)
997	35.694084700	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=323/17153, ttl=3 (no response found!)
1000	35.701523700	10.149.252.37	192.168.11.109	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
1001	35.703045100	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=324/17409, ttl=3 (no response found!)
1002	35.707115500	10.149.252.37	192.168.11.109	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
1004	35.708161100	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=325/17665, ttl=3 (no response found!)
1005	35.712695300	10.149.252.37	192.168.11.109	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
1244	45.753792100	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=326/17921, ttl=4 (no response found!)
1245	45.760493900	89.223.47.5	192.168.11.109	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
1246	45.770912000	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=327/18177, ttl=4 (no response found!)
1247	45.795418400	89.223.47.5	192.168.11.109	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
1248	45.798334000	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=328/18433, ttl=4 (no response found!)
1249	45.809242400	89.223.47.5	192.168.11.109	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
1513	55.841570000	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=329/18689, ttl=5 (no response found!)
1515	55.859558700	185.0.12.221	192.168.11.109	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
1516	55.861168000	192.168.11.109	87.236.16.90	ICMP	106	Echo (ping) request id=0x0001, seq=330/18945, ttl=5 (no response found!)

3. Чем отличаются ICMP-пакеты, генерируемые tracert, от ICMP-пакетов ping?

Ping всегда шлёт ICMP Echo Request и ждёт Echo Reply. Tracert использует ICMP Echo Request с разным TTL и анализирует:

- ICMP Time Exceeded от промежуточных маршрутизаторов.
- ICMP Echo Reply от конечного узла.

То есть ping проверяет доступность узла, а tracert строит маршрут до него.

4. Чем отличаются ICMP reply от ICMP error и зачем нужны оба?

ICMP reply – отклик от целевого хоста, подтверждающий, что он доступен.

ICMP error – приходит от маршрутизаторов, когда TTL истекает. Эти пакеты нужны для определения маршрута.

Оба типа позволяют tracert:

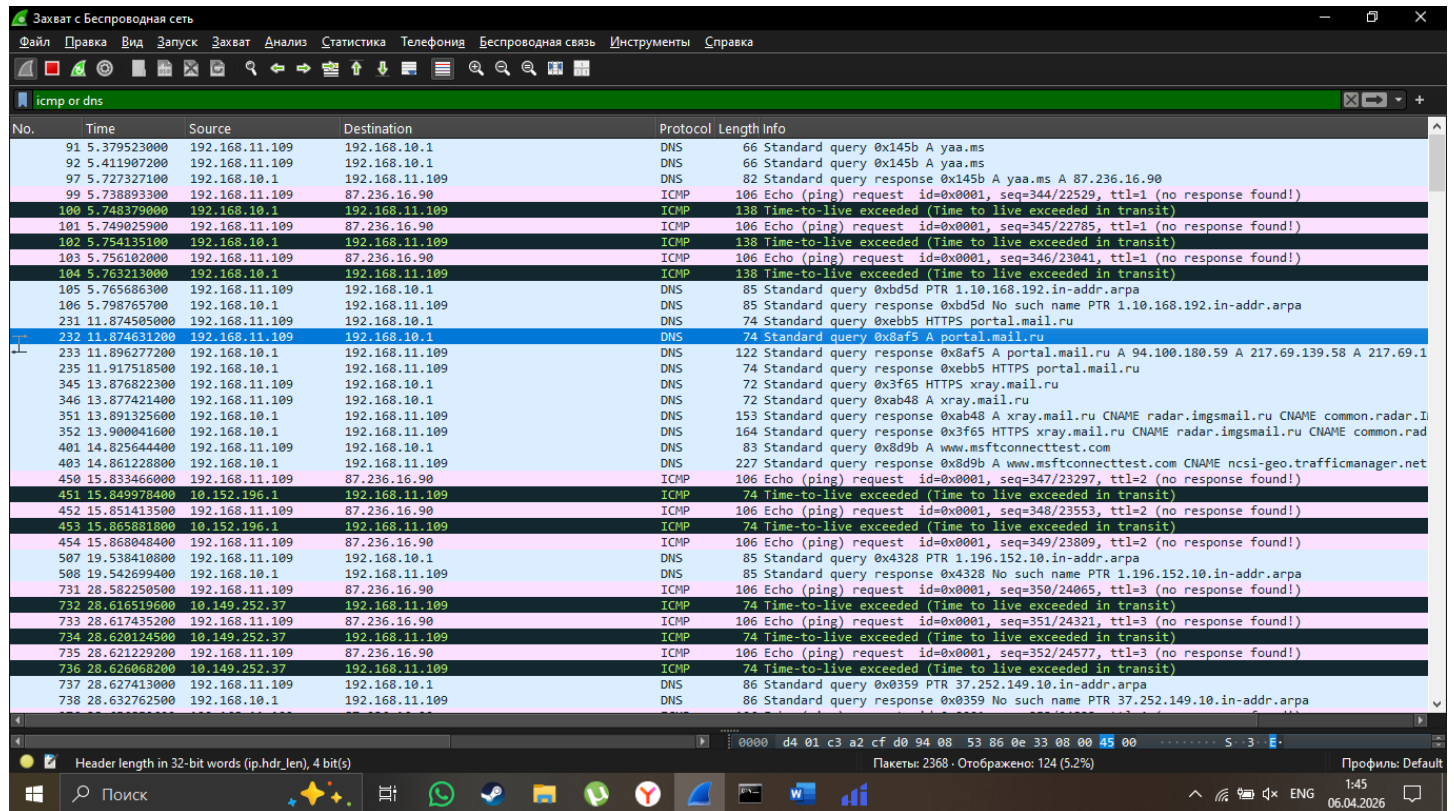
- Узнать IP каждого промежуточного маршрутизатора (через error).
- Подтвердить достижение конечного узла (через reply).

```
▼ Internet Control Message Protocol
  Type: Echo (ping) request (8)
  Code: 0
  Checksum: 0xf6bc [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 322 (0x0142)
  Sequence Number (LE): 16897 (0x4201)
```

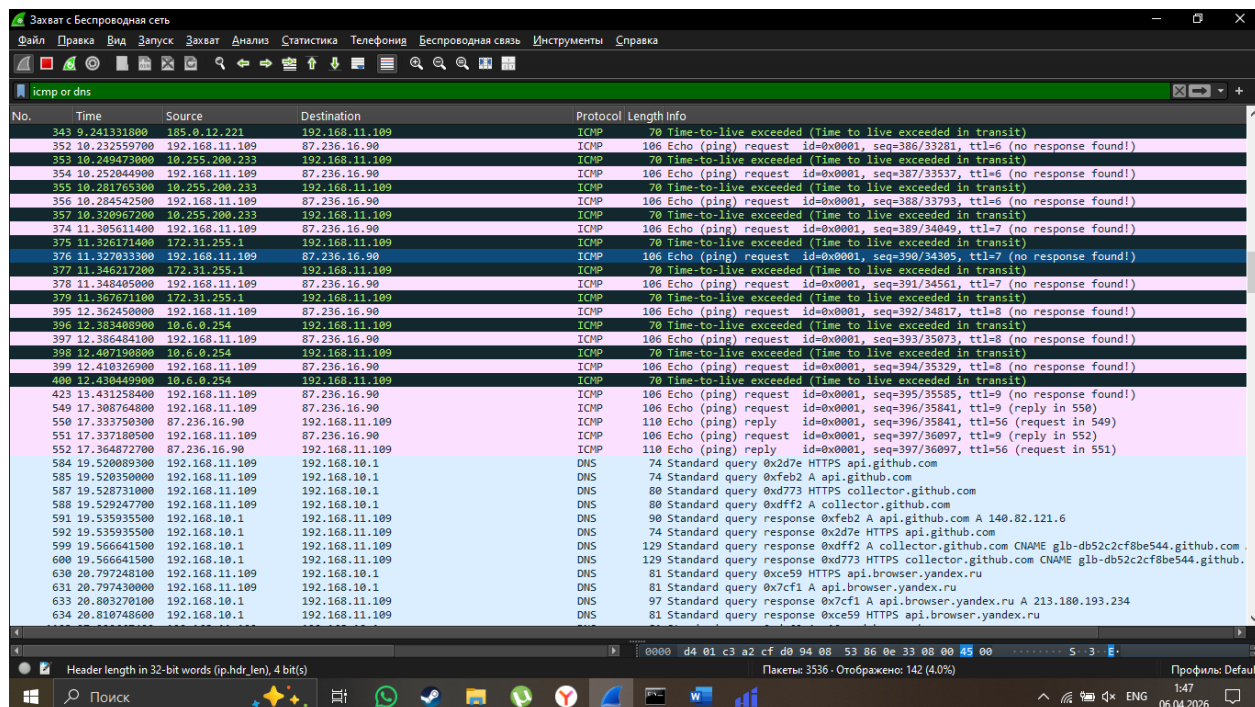
5. Что изменится в работе tracert, если убрать ключ -d? Какой трафик будет генерироваться дополнительно?

Ключ -d отключает обратное разрешение IP-адресов в доменные имена. Без -d tracert будет пытаться разрешить IP-адреса в имена хостов (через DNS). Это приведёт к дополнительному DNS-трафику, так как каждый IP будет запрашиваться у DNS-сервера.

Вот окно wireshark без ключа -d:



Вот окно wireshark с ключом -d:

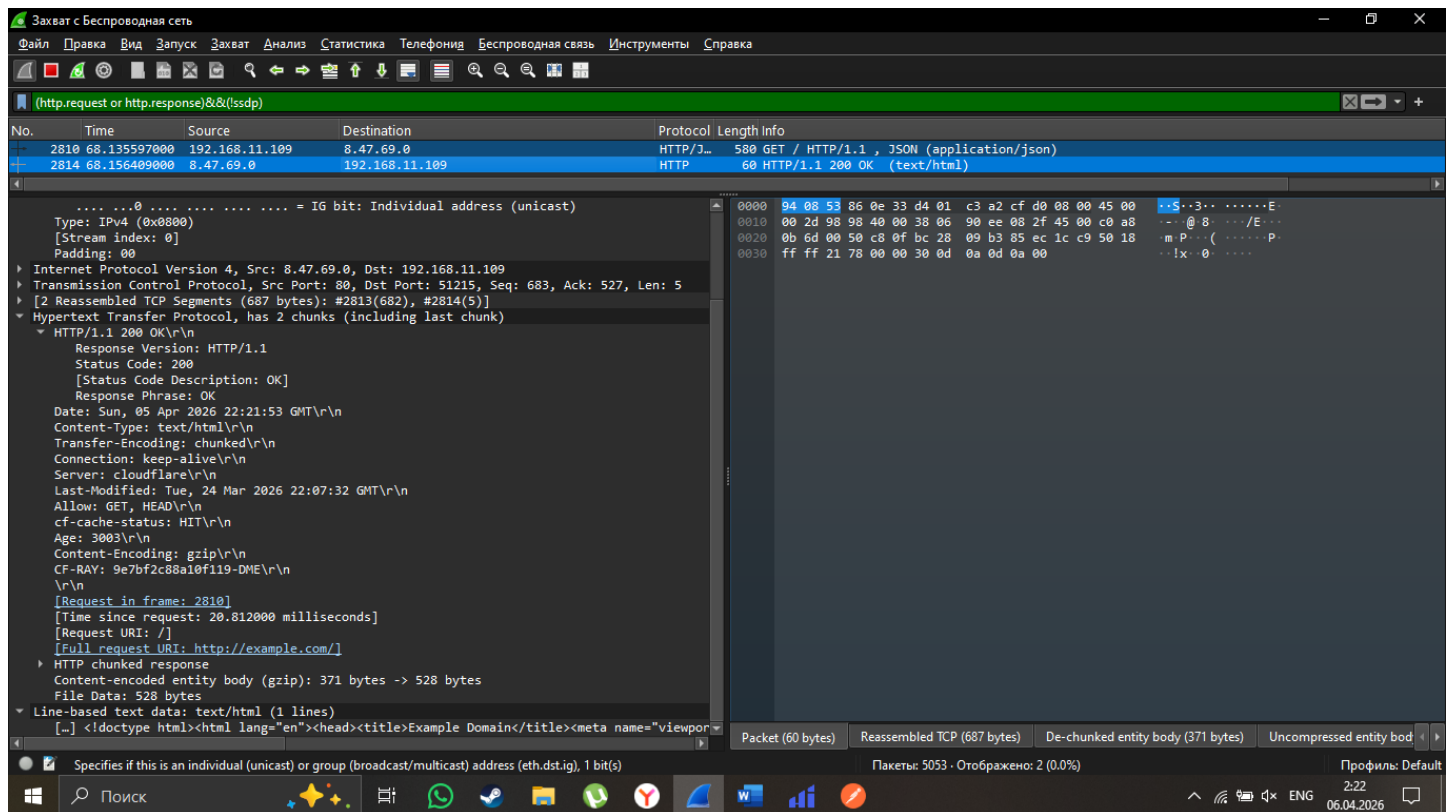


Анализ HTTP-трафика

Выполнение

Запустим анализ в Wireshark и перейдём на сайт yaa.ms. К сожалению, сколько раз не обновляй мы не можем получить ответ 304. Поэтому воспользуемся сайтом, который точно обладает такой возможностью, а именно сайтом example.com.

Сначала просто зайдём на сайт example.com.



Заметим, что все отработало как надо, мы получаем ответ 200.

Теперь попробуем обновить страницу и посмотрим, что будет.

No.	Time	Source	Destination	Protocol	Length	Info
116	2.029677600	192.168.11.109	8.6.112.0	HTTP/JSON	630	GET / HTTP/1.1 , JSON (application/json)
119	2.048130000	8.6.112.0	192.168.11.109	HTTP	313	HTTP/1.1 304 Not Modified

Frame 119: Packet, 313 bytes on wire (2504 bits), 313 bytes captured (2504 bits) on interface
Ethernet II, Src: Routerboardc_a2:cf:d0 (d4:01:c3:a2:cf:d0), Dst: LiteonTechno_86:0e:33 (94:08:53:86:0e:33)
Destination: LiteonTechno_86:0e:33 (94:08:53:86:0e:33)
.....0..... = LG bit: Globally unique address (factory default)
.....0..... = IG bit: Individual address (unicast)
Source: Routerboardc_a2:cf:d0 (d4:01:c3:a2:cf:d0)
.....0..... = LG bit: Globally unique address (factory default)
.....0..... = IG bit: Individual address (unicast)
Type: IPv4 (0x0800)
[Stream index: 0]
Internet Protocol Version 4, Src: 8.6.112.0, Dst: 192.168.11.109
Transmission Control Protocol, Src Port: 80, Dst Port: 55330, Seq: 1, Ack: 578, Len: 259
Hypertext Transfer Protocol
HTTP/1.1 304 Not Modified
Response Version: HTTP/1.1
Status Code: 304
[Status Code Description: Not Modified]
Response Phrase: Not Modified
Date: Sun, 05 Apr 2026 22:30:43 GMT
Connection: keep-alive
Allow: GET, HEAD
Age: 3534
Server: cloudflare
Last-Modified: Tue, 24 Mar 2026 22:07:32 GMT
etag: "69c30b24-210"
cf-cache-status: HIT
CF-RAY: 9e7bffbce8859da7-DME
[Request in frame: 116]
[Time since request: 18.452400 milliseconds]
[Request URI: /]
[Full request URI: http://example.com/]

0000 94 08 53 86 0e 33 d4 01 c3 a2 cf d0 08 00 45 00 ..S..3.....E
0010 01 2b 71 0e 40 00 38 06 8c a3 08 06 70 00 c0 a8 +q@8.....p..
0020 0b 6d 00 50 d8 22 7e d5 3f 6b 39 dd b9 b4 50 18 mP"~?k9..P..
0030 ff ff 09 e3 00 00 48 54 54 50 2f 31 2e 31 20 33HTP/1.1.3
0040 30 34 20 4e 6f 74 20 4d 6f 64 69 66 69 65 64 0d 04 Not Modified
0050 0a 44 61 74 65 3a 20 53 75 6e 2c 20 30 35 20 41 ..Date: Sun, 05 A
0060 70 72 20 32 30 32 36 20 32 32 3a 33 30 3a 34 33 pr 2026 22:30:43
0070 20 47 4d 54 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e GMT Connection
0080 3a 20 6b 65 65 70 2d 61 6c 69 76 65 0d 0a 41 6c : keep-alive..Al
0090 6c 6f 77 3a 20 47 45 54 2c 20 48 45 41 44 0d 0a low: GET, HEAD..
00a0 41 67 65 3a 20 33 35 33 34 0d 0a 53 65 72 76 65 Age: 3534. Serve
00b0 72 3a 20 63 6c 6f 75 64 66 6c 61 72 65 0d 0a 4c r: cloud flare..L
00c0 61 73 74 2d 4d 6f 64 69 66 69 65 64 3a 20 54 75 ast-Modified: Tu
00d0 65 2c 20 32 34 20 4d 61 72 20 32 30 32 36 20 32 e, 24 Ma r 2026 2
00e0 32 3a 30 37 3a 33 32 20 47 4d 54 0d 0a 65 74 61 2:07:32 GMT eta
00f0 67 3a 20 22 36 39 63 33 30 62 32 34 2d 32 31 30 g: "69c3 0b24-210
0100 22 0d 0a 63 66 2d 63 61 63 68 65 2d 73 74 61 74 ".cf-ca che-stat
0110 75 73 3a 20 48 49 54 0d 0a 43 46 2d 52 41 59 3a us: HIT CF-RAY:
0120 20 39 65 37 62 66 66 62 63 65 38 38 35 39 64 61 9e7bffb ce8859da
0130 37 2d 44 4d 45 0d 0a 0d 0a 7-DME ..

Hypertext Transfer Protocol
HTTP/1.1 304 Not Modified
Date: Sun, 05 Apr 2026 22:30:43 GMT
Connection: keep-alive
Allow: GET, HEAD
Age: 3534
Server: cloudflare
Last-Modified: Tue, 24 Mar 2026 22:07:32 GMT
etag: "69c30b24-210"
cf-cache-status: HIT
CF-RAY: 9e7bffbce8859da7-DME
[Request in frame: 116]
[Time since request: 18.452400 milliseconds]
[Request URI: /]
[Full request URI: http://example.com/]

Hypertext Transfer Protocol
GET / HTTP/1.1
If-Modified-Since: Sun, 05 Apr 2026 22:28:05 GMT
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQI0jE1NSwidXN1cm5hbWUiOiJmb3Nfc6luZ3Vpbm9zX211X2xhX3Zhb19hX21hc2NhcisImhdCI6MTczNzc0Njc2MywiZXhwIjoxNz00MzMTMzfq
User-Agent: PostmanRuntime/7.43.0
Accept: */*
Postman-Token: 6d990c34-2598-4a3a-a42e-63f9626cb0f7
Host: example.com
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 45
[Response in frame: 119]
[Full request URI: http://example.com/]
File Data: 45 bytes

Заметим, что мы получаем совсем другую ситуацию. Здесь у нас получилось отправить условный GET-запрос. И мы получаем ответ 304 от сервера. Это можно понять по появившимся полям Last-Modified и If-Modified-Since.

Анализ ARP-трафика

Выполнение

Для начала очистим ARP-таблицу с помощью команды:

netsh interface ip delete arpcache

```
C:\Windows\system32>netsh interface ip delete arpcache
OK.
```

Получим вот такую ARP-таблицу:

```
C:\Windows\system32>arp -a

Интерфейс: 192.168.11.109 --- 0x4
    адрес в Интернете      Физический адрес      Тип
    192.168.10.1            d4-01-c3-a2-cf-d0      динамический
    224.0.0.22              01-00-5e-00-00-16      статический
```

После удаления кэша браузера отправимся на сайт yaa.ms и увидим новую запись в ARP-таблице.

```
C:\Windows\system32>arp -a

Интерфейс: 192.168.11.109 --- 0x4
    адрес в Интернете      Физический адрес      Тип
    192.168.10.1            d4-01-c3-a2-cf-d0      динамический
    224.0.0.22              01-00-5e-00-00-16      статический
    224.0.0.251             01-00-5e-00-00-fb      статический
    224.0.0.252             01-00-5e-00-00-fc      статический
```

Заметим, что это вообще не похоже на IP адрес сайта, на который мы перешли. А всё, потому что MAC-адреса используются только в локальной сети. Мы не сможем увидеть ARP-запрос, который узнаёт MAC-адрес нашего сайта, так как его и вовсе нет. Но мы видим IP-адрес 192.168.1.1. Вероятнее всего это IP нашего маршрутизатора, который как раз таки и взялся в дальнейшем уже за поиск того сайта, на который мы перешли.

No.	Time	Source	Destination	Protocol	Length	Info
245	5.398109400	Routerboardc_a2:cf:...	Broadcast	ARP	64	who has 192.168.11.22? Tell 192.168.10.1
325	6.524541600	Routerboardc_a2:cf:...	Broadcast	ARP	64	who has 192.168.10.173? Tell 192.168.10.1
616	9.486819000	LiteonTechno_86:0e:...	Broadcast	ARP	42	who has 192.168.10.1? Tell 192.168.11.109
617	9.494216200	Routerboardc_a2:cf:...	LiteonTechno_86:0e:33	ARP	64	192.168.10.1 is at d4:01:c3:a2:cf:d0
732	14.101995100	Routerboardc_a2:cf:...	Broadcast	ARP	64	who has 192.168.11.165? Tell 192.168.10.1
756	14.511578500	Routerboardc_a2:cf:...	Broadcast	ARP	64	who has 192.168.11.165? Tell 192.168.10.1
761	14.613978000	Routerboardc_a2:cf:...	Broadcast	ARP	64	who has 192.168.11.165? Tell 192.168.10.1
762	14.818774400	Routerboardc_a2:cf:...	Broadcast	ARP	64	who has 192.168.11.165? Tell 192.168.10.1
763	14.921200200	Routerboardc_a2:cf:...	Broadcast	ARP	64	who has 192.168.11.165? Tell 192.168.10.1
764	14.921424200	Routerboardc_a2:cf:...	Broadcast	ARP	64	who has 192.168.11.165? Tell 192.168.10.1

Frame 616: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \D Ethernet II, Src: LiteonTechno_86:0e:33 (94:08:53:86:0e:33), Dst: Broadcast (ff:ff:ff:ff:ff:ff) Destination: Broadcast (ff:ff:ff:ff:ff:ff) Source: LiteonTechno_86:0e:33 (94:08:53:86:0e:33) Type: ARP (0x0806) [Stream index: 17] Address Resolution Protocol (request) Hardware type: Ethernet (1) Protocol type: IPv4 (0x0800) Hardware size: 6 Protocol size: 4 Opcode: request (1) Sender MAC address: LiteonTechno_86:0e:33 (94:08:53:86:0e:33) Sender IP address: 192.168.11.109 Target MAC address: 00:00:00 00:00:00 (00:00:00:00:00:00) Target IP address: 192.168.10.1	<pre> 0000 ff ff ff ff ff 94 08 53 86 0e 33 08 06 00 01 S 3 ... 0010 08 00 06 04 00 01 94 08 53 86 0e 33 c0 a8 0b 6d S 3 ...m 0020 00 00 00 00 00 00 c0 a8 0a 01 </pre>
--	---

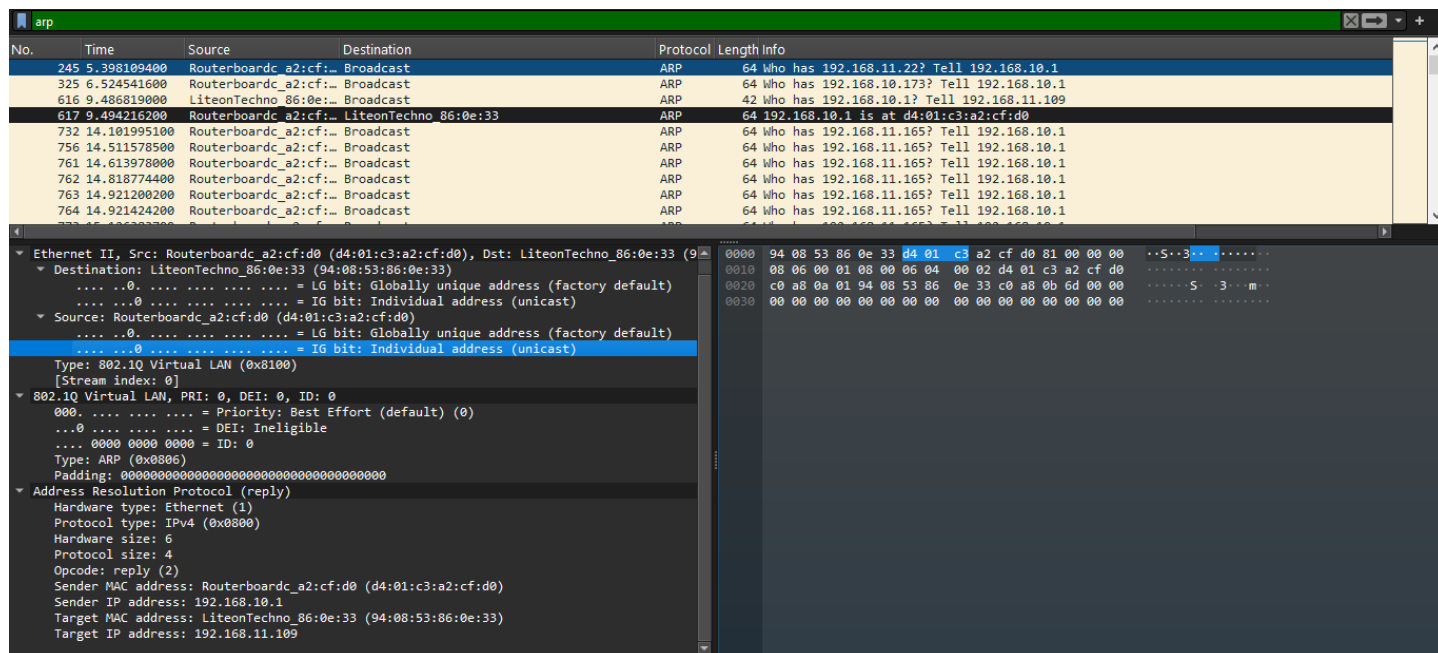
Ответы на вопросы:

1. Какие MAC-адреса присутствуют в захваченных пакетах ARP-протокола? Что означают эти адреса? Какие устройства они идентифицируют?

В ARP-пакетах мы увидим два типа MAC-адресов:

- MAC-адрес отправителя запроса – адрес нашего компьютера. Он используется в поле Sender MAC-address
- MAC-адрес искомого устройства:
 - В ARP-запросе (who-has) поле Target MAC Address будет заполнено нулями, потому что он ещё известен.
 - В ARP-ответе (is-at) это будет MAC-адрес шлюза/маршрутизатора, провайдера или другого узла локальной сети, связанного с IP, на который отправляется запрос.

2. Какие MAC-адреса присутствуют в захваченных HTTP-пакетах и что означают эти адреса? Что означают эти адреса? Какие устройства они идентифицируют?



The image shows a Wireshark packet capture of an ARP process. The top pane displays a list of packets, including several ARP requests (No. 245, 325, 616, 617, 732, 756, 761, 762, 763, 764) and one ARP reply (No. 617). The bottom pane shows a detailed view of the selected ARP reply packet (No. 617). The packet details include:

- Ethernet II, Src: Routerboardc_a2:cf:d0 (d4:01:c3:a2:cf:d0), Dst: LiteonTechno_86:0e:33 (94:08:53:86:0e:33)
- Type: 802.1Q Virtual LAN (0x8100)
- 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 0
- Type: ARP (0x0806)
- Address Resolution Protocol (reply)
- Hardware type: Ethernet (1)
- Protocol type: IPv4 (0x0800)
- Hardware size: 6
- Protocol size: 4
- Opcode: reply (2)
- Sender MAC address: Routerboardc_a2:cf:d0 (d4:01:c3:a2:cf:d0)
- Sender IP address: 192.168.10.1
- Target MAC address: LiteonTechno_86:0e:33 (94:08:53:86:0e:33)
- Target IP address: 192.168.11.109

The packet bytes pane shows the raw data of the ARP reply, including the Ethernet II header, the 802.1Q VLAN tag, and the ARP payload.

HTTP работает поверх TCP/IP и Ethernet. В Ethernet-заголовке каждого HTTP-пакета указывается:

- MAC-адрес источника – это MAC-адрес нашего компьютера
- MAC-адрес назначения – это обычно MAC-адрес ближайшего маршрутизатора/шлюза, через который трафик пойдёт в Интернет.

MAC-адреса веб-сайта, на который мы заходим, мы не увидим, потому что MAC-адреса используются только внутри локальной сети.

3. Для чего ARP-запрос содержит IP-адрес источника?

ARP-запроса содержит IP-адрес источника, чтобы:

- Получатель запроса (тот, чей IP адрес запрашивается) мог записать в свою ARP-таблицу соответствие, и тем самым сократить количество ARP-запросов в будущем.
- Получатель понимал, кто запрашивает – это нужно для формирования ARP-запроса-ответа.

IP-адрес источника нужен для обратной связи и корректного построения локальной маршрутизации.

Вывод

В ходе лабораторной работы с помощью программы Wireshark был выполнен анализ передачи сетевых пакетов, а также описана структура протоколов DNS, ICMP, IP, ARP и HTTP. Я изучил, какие пакеты формируются при работе утилит ping и traceroute, и какую информацию они несут. Дополнительно был проведён анализ трафика HTTP-запросов и влияния на него кэширования данных.

Кэширование также воздействует на функционирование DNS: в процессе работы требовалось очищать кэш и наблюдать за поведением DNS-запросов. Затем был рассмотрен трафик при выполнении ARP-запросов, для чего потребовалось очистить ARP-таблицу. В итоге я понял, что передача данных по сети — это сложный механизм, включающий взаимодействие множества протоколов и интерфейсов.